



Students Social Media Addiction

Khushi

Project Background



-
1. Social media has become an important part of students' daily lives.
 2. It helps people communicate, learn, and stay connected with others.
 3. Excessive use of social media can negatively affect: - Mental health - Sleep quality - Academic performance - Personal relationships
 4. Many students spend long hours on social media platforms, which may lead to: - Stress - Anxiety - Lack of concentration - Reduced productivity - Sleep disturbances
 5. Overuse of social media can also affect emotional well-being and increase personal conflicts.
 6. This project analyzes students' social media usage patterns and addiction levels.
 7. Different factors studied in this project include: - Daily usage hours - Sleep duration - Mental health score - Academic impact - Social conflicts
 8. Machine Learning models are used to predict addiction scores and identify the most influential factors.
 9. The project also demonstrates practical applications of: - Data Analysis - Data Visualization - Feature Engineering - Predictive Modeling

Project Pipeline

1. Imported important libraries such as: Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn
2. Loaded the dataset using Pandas.
3. Explored the dataset by checking: Shape of dataset, Column names, Data types, Missing values
4. Performed data cleaning and preprocessing.
5. Detected and removed outliers using the IQR method.
6. Identified numeric and categorical columns.
7. Performed Exploratory Data Analysis (EDA) using: Histograms, Boxplots, Countplots, Correlation Heatmap
8. Encoded categorical columns using Label Encoding.
9. Created new features using Feature Engineering: Usage_Sleep_Ratio, Mental_Conflict_Score
10. Split the dataset into: Training data, Testing data
11. Applied Feature Scaling using StandardScaler.
12. Trained Machine Learning models: Linear Regression, Random Forest Regressor
13. Predicted addiction scores using trained models.
14. Evaluated model performance using: MAE, MSE, RMSE, R^2 Score
15. Compared model performances.
16. Visualized: Actual vs Predicted values, Feature Importance, Model Comparison
17. Identified the best-performing model based on evaluation metrics.
18. Concluded the project by analyzing important factors affecting social media addiction among students

IMPORT LIBRARIES

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
# Ignore warnings
import warnings
warnings.filterwarnings("ignore")

print("Libraries imported successfully!")
```

LOAD THE DATASET

```
# Read CSV file
df = pd.read_csv("Students Social Media Addiction.csv")
```

Basic Info

```
# Display the first few rows of the dataset  
df.head(5)
```

	Student_ID	Age	Gender	Academic_Level	Country	Avg_Daily_Usage_Hours	Most_Used_Platform	Affects_Academic_Performance	Sleep_Hours_Per_Night	Mental_H
0	1	19	Female	Undergraduate	Bangladesh	5.2	Instagram	Yes	6.5	
1	2	22	Male	Graduate	India	2.1	Twitter	No	7.5	
2	3	20	Female	Undergraduate	USA	6.0	TikTok	Yes	5.0	
3	4	18	Male	High School	UK	3.0	YouTube	No	7.0	
4	5	21	Male	Graduate	Canada	4.5	Facebook	Yes	6.0	

```
# Shape means rows and columns  
print("\nShape of Dataset:")  
print(df.shape)
```

```
Shape of Dataset:  
(705, 13)
```

The dataset contains information about students' social media usage, mental health, sleep patterns, and addiction scores.

```
# Show column names
print("\nColumn Names:")
print(df.columns)
```

```
Column Names:
Index(['Student_ID', 'Age', 'Gender', 'Academic_Level', 'Country',
      'Avg_Daily_Usage_Hours', 'Most_Used_Platform',
      'Affects_Academic_Performance', 'Sleep_Hours_Per_Night',
      'Mental_Health_Score', 'Relationship_Status',
      'Conflicts_Over_Social_Media', 'Addicted_Score'],
      dtype='object')
```

```
# Check data types
print("\nData Types:")
print(df.dtypes)
```

```
Data Types:
Student_ID          int64
Age                 int64
Gender              object
Academic_Level      object
Country             object
Avg_Daily_Usage_Hours  float64
Most_Used_Platform  object
Affects_Academic_Performance  object
Sleep_Hours_Per_Night  float64
Mental_Health_Score  int64
Relationship_Status  object
Conflicts_Over_Social_Media  int64
Addicted_Score      int64
dtype: object
```

MISSING VALUES

```
# Check missing values  
print("\nMissing Values:")  
print(df.isnull().sum())
```

```
Missing Values:  
Student_ID          0  
Age                 0  
Gender              0  
Academic_Level      0  
Country             0  
Avg_Daily_Usage_Hours 0  
Most_Used_Platform  0  
Affects_Academic_Performance 0  
Sleep_Hours_Per_Night 0  
Mental_Health_Score 0  
Relationship_Status 0  
Conflicts_Over_Social_Media 0  
Addicted_Score      0  
dtype: int64
```

No missing values were found in the dataset, indicating clean and complete data.

OUTLIER HANDLING USING IQR METHOD

```
# Select only numeric columns
num_cols = df.select_dtypes(include=np.number).columns
# Remove target column from outlier handling
num_cols = num_cols.drop("Addicted_Score")
# Loop through each numeric column
for col in num_cols:
    Q1 = df[col].quantile(0.25)
    # Calculate Q3 (75th percentile)
    Q3 = df[col].quantile(0.75)
    # Calculate IQR
    IQR = Q3 - Q1
    # Lower limit
    lower_limit = Q1 - 1.5 * IQR
    # Upper limit
    upper_limit = Q3 + 1.5 * IQR
    # Before removing outliers
    old_shape = df.shape[0]
    # Keep only normal values
    df = df[
        (df[col] >= lower_limit) &
        (df[col] <= upper_limit)]
    # After removing outliers
    new_shape = df.shape[0]
    print(f"{col}")
    print("Rows before removing outliers :", old_shape)
    print("Rows after removing outliers :", new_shape)
    print("-" * 50)
```

Student_ID

Rows before removing outliers : 705

Rows after removing outliers : 705

Age

Rows before removing outliers : 705

Rows after removing outliers : 705

Avg_Daily_Usage_Hours

Rows before removing outliers : 705

Rows after removing outliers : 702

Sleep_Hours_Per_Night

Rows before removing outliers : 702

Rows after removing outliers : 702

Mental_Health_Score

Rows before removing outliers : 702

Rows after removing outliers : 702

Conflicts_Over_Social_Media

Rows before removing outliers : 702

Rows after removing outliers : 702

STATISTICAL SUMMARY

```
print("\nStatistical Summary:")
print(df[num_cols].describe())
```

```
Statistical Summary:
```

	Student_ID	Age	Avg_Daily_Usage_Hours	Sleep_Hours_Per_Night \
count	702.000000	702.000000	702.000000	702.000000
mean	353.096866	20.659544	4.913533	6.875926
std	203.541208	1.397793	1.239162	1.116842
min	1.000000	18.000000	2.000000	4.000000
25%	177.250000	19.250000	4.100000	6.000000
50%	352.500000	21.000000	4.800000	6.900000
75%	529.750000	22.000000	5.800000	7.700000
max	705.000000	24.000000	8.300000	9.600000

	Mental_Health_Score	Conflicts_Over_Social_Media	Addicted_Score
count	702.000000	702.000000	702.000000
mean	6.226496	2.850427	6.435897
std	1.100504	0.951982	1.575768
min	4.000000	0.000000	3.000000
25%	5.000000	2.000000	5.000000
50%	6.000000	3.000000	7.000000
75%	7.000000	4.000000	8.000000
max	8.000000	5.000000	9.000000

The statistical summary provides information about the distribution, average values, minimums, maximums, and spread of numerical features.

NUMERIC AND CATEGORICAL COLUMNS

The dataset contains both numerical and categorical features which were handled separately during preprocessing.

```
num_cols = df.select_dtypes(include=np.number).columns
cat_cols = df.select_dtypes(include="object").columns

print("Numeric Columns:")
print(num_cols)

print()

print("Categorical Columns:")
print(cat_cols)
```

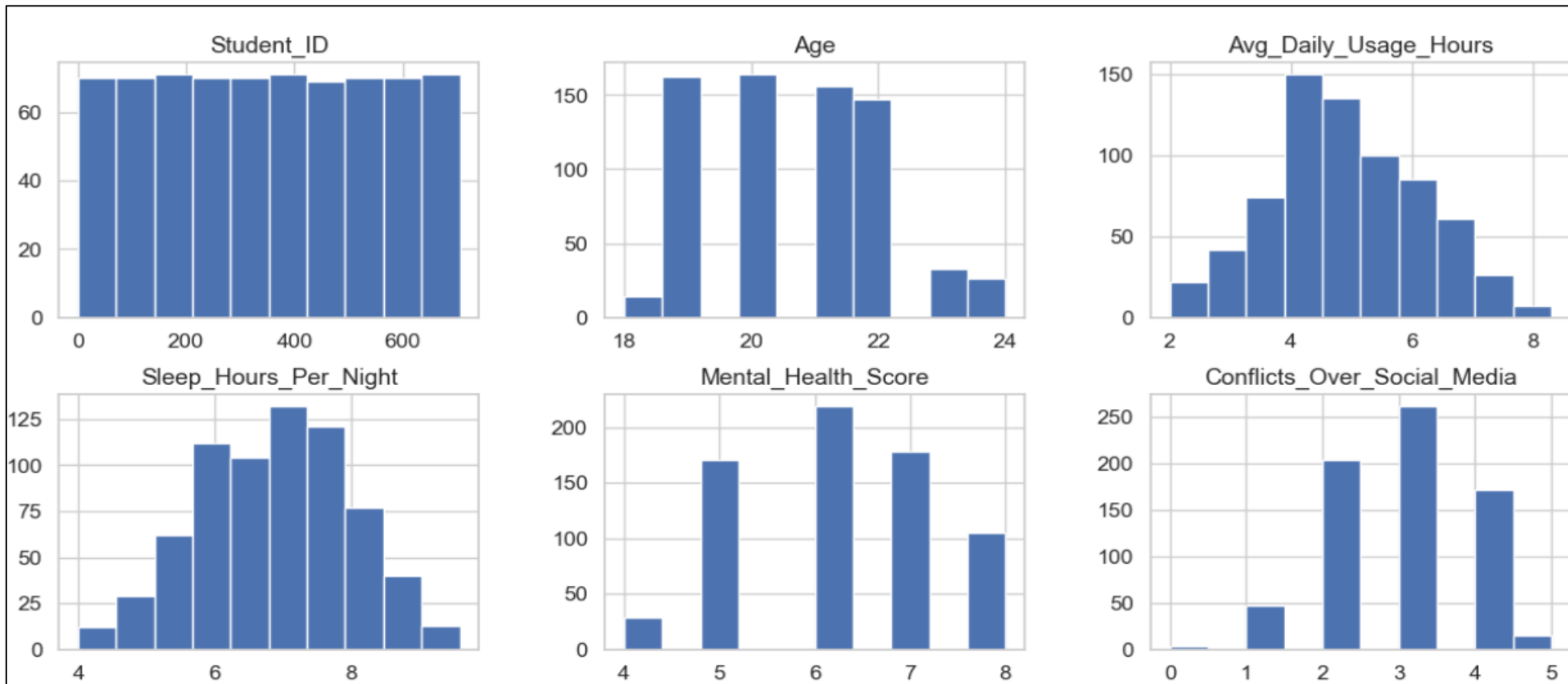
Numeric Columns:
Index(['Student_ID', 'Age', 'Avg_Daily_Usage_Hours', 'Sleep_Hours_Per_Night',
 'Mental_Health_Score', 'Conflicts_Over_Social_Media', 'Addicted_Score'],
 dtype='object')

Categorical Columns:
Index(['Gender', 'Academic_Level', 'Country', 'Most_Used_Platform',
 'Affects_Academic_Performance', 'Relationship_Status'],
 dtype='object')

HISTOGRAMS

```
# Histogram shows distribution of data
df.hist(figsize=(15,10))
plt.suptitle("Histograms of Numeric Columns")
plt.show()
```

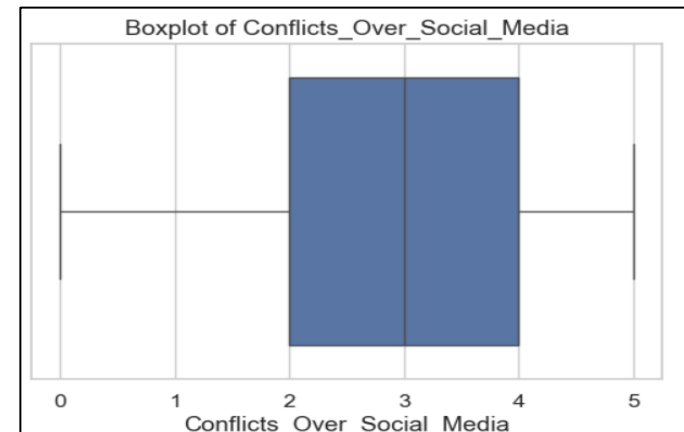
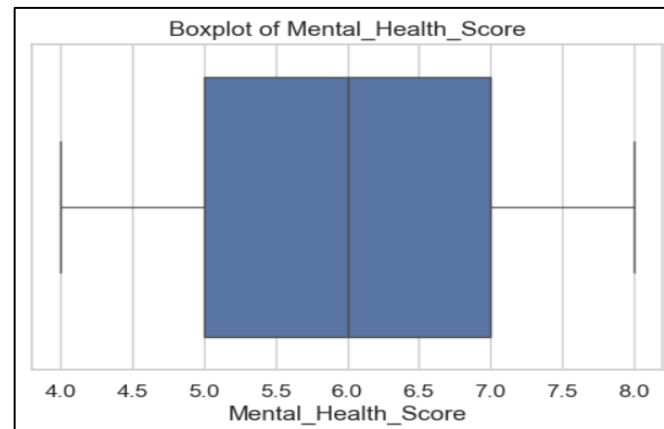
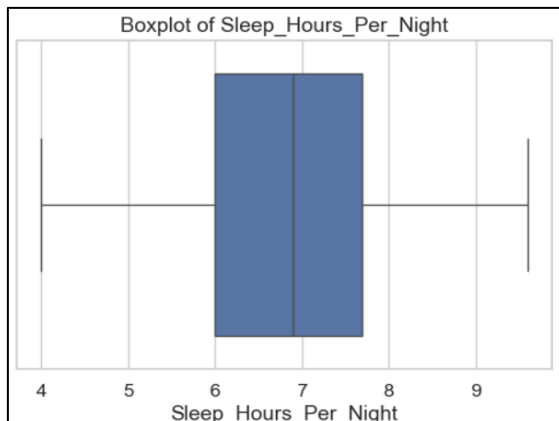
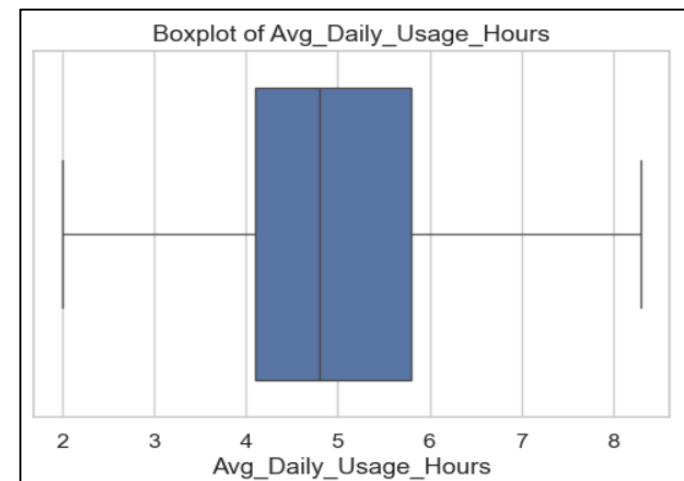
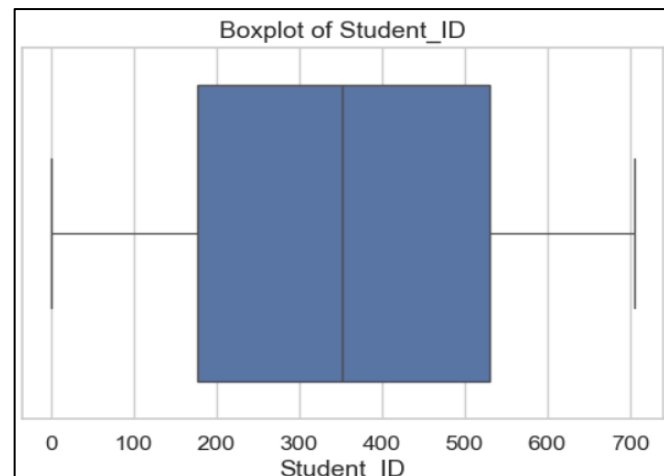
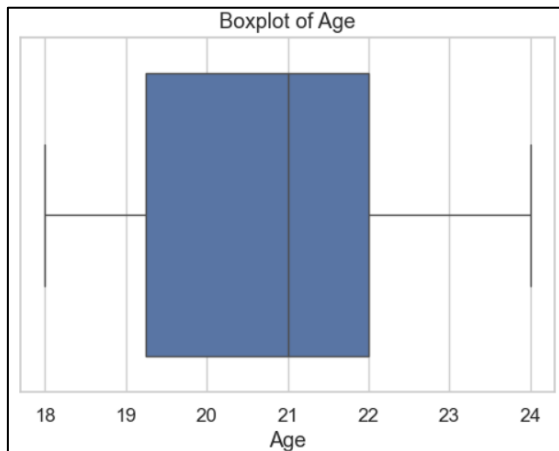
Histograms show the distribution of numerical features such as age, daily usage hours, sleep hours, and addiction score.



BOXPLOTS

```
for col in num_cols:
    plt.figure(figsize=(6,4))
    sns.boxplot(x=df[col])
    plt.title(f"Boxplot of {col}")
    plt.show()
```

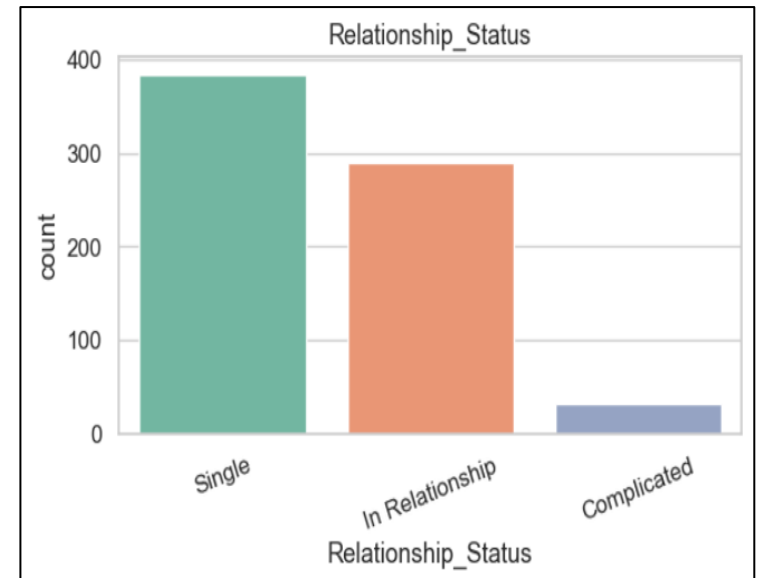
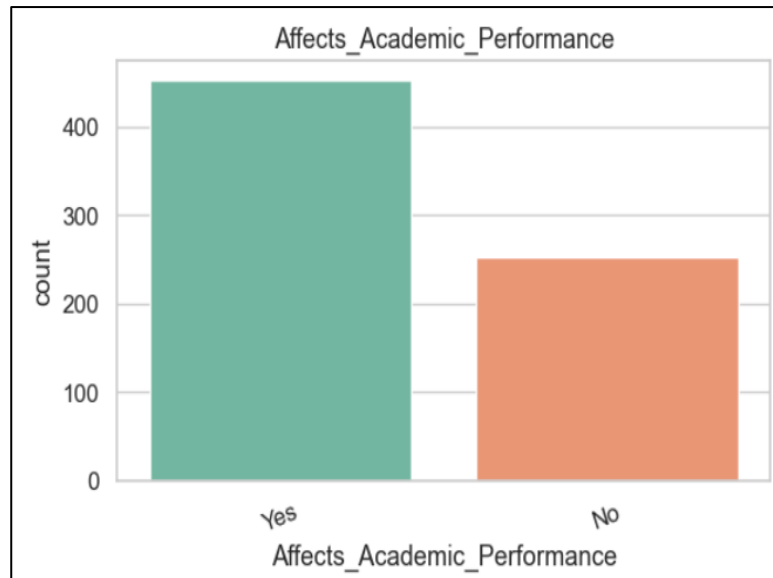
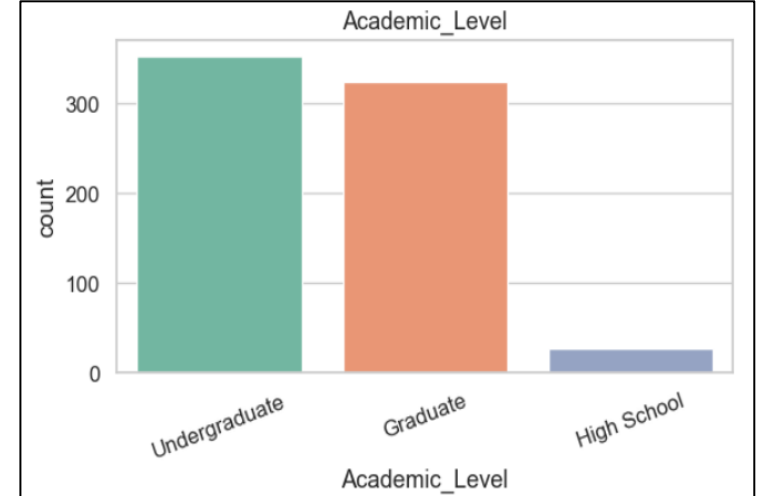
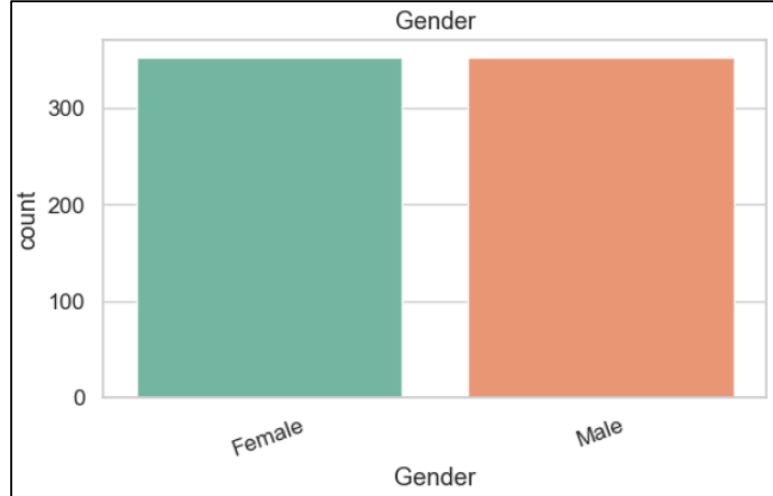
Boxplots help visualize data spread and identify possible outliers in numerical columns.



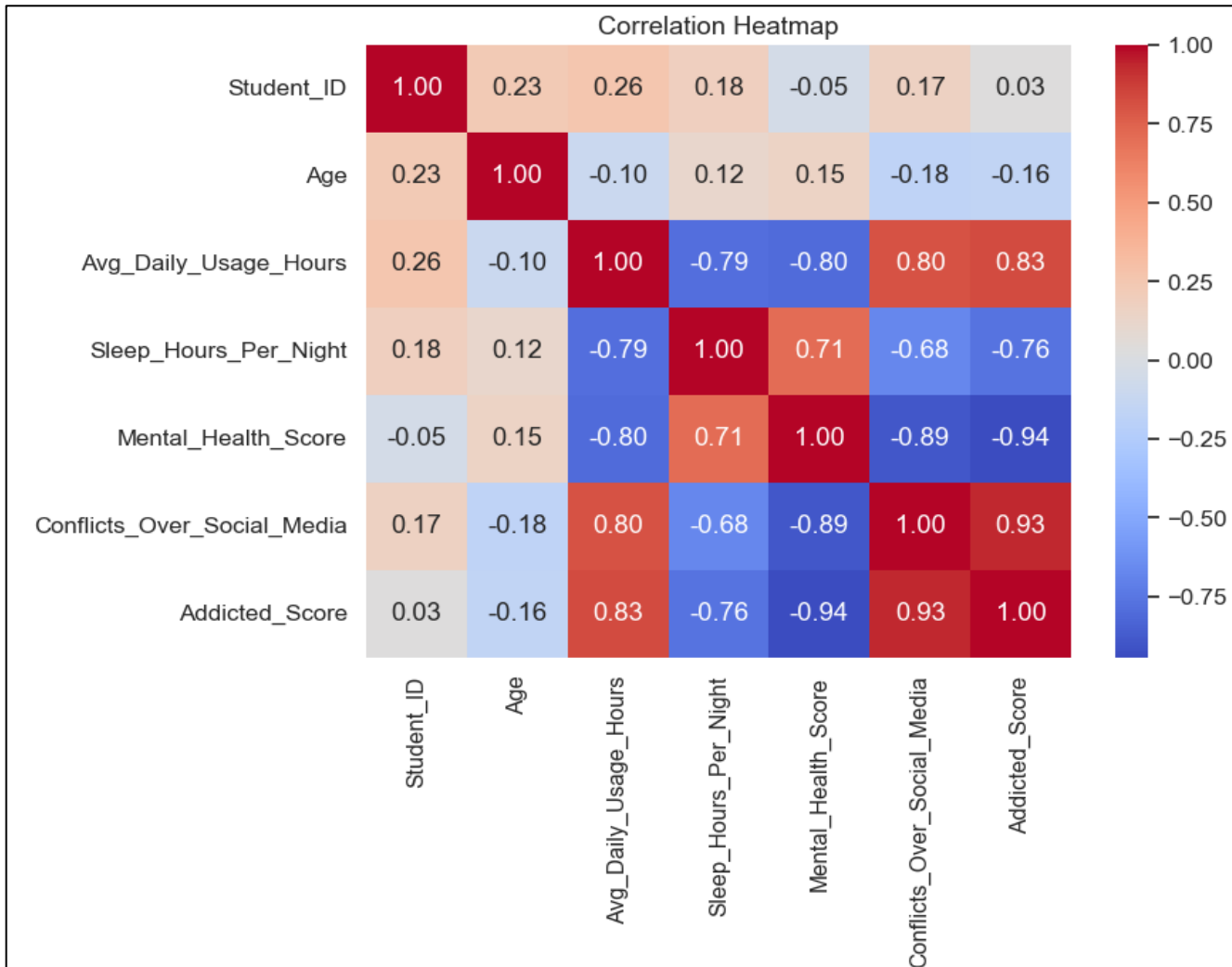
COUNTPLOTS

```
for col in cat_cols:
    # Skip columns with too many unique values
    if df[col].nunique() > 10:
        continue
    # Smaller figure size
    plt.figure(figsize=(7,4))
    # Sort categories
    order = df[col].value_counts().index
    # Create countplot
    sns.countplot(x=df[col], order=order, palette="Set2" )
    plt.title(f"{col}", fontsize=13)
    plt.xticks(rotation=20)
    # Better spacing
    plt.tight_layout()
    plt.show()
```

Countplots show the frequency distribution of categorical features such as gender, academic level, and most used platform.



CORRELATION HEATMAP



```
# Correlation tells relationship between columns
corr = df[num_cols].corr()
plt.figure(figsize=(8,6))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```

The heatmap shows relationships between numerical features and indicates that mental health and conflicts have strong influence on addiction score.

CREATE FEATURES AND TARGET

```
X = df.drop("Addicted_Score", axis=1)

y = df["Addicted_Score"]
```

Features and target variables were separated, and categorical columns were converted into numerical format using Label Encoding.

LABEL ENCODING

```
# Create encoder
le = LabelEncoder()
# Apply encoding on every categorical column
for col in cat_cols:
    df[col] = le.fit_transform(df[col])
print("\nEncoded Dataset:")
print(df.head())
```

Encoded Dataset:

	Student_ID	Age	Gender	Academic_Level	Country	Avg_Daily_Usage_Hours	\
0	1	19	0	2	10	5.2	
1	2	22	1	0	39	2.1	
2	3	20	0	2	102	6.0	
3	4	18	1	1	101	3.0	
4	5	21	1	0	18	4.5	

	Most_Used_Platform	Affects_Academic_Performance	Sleep_Hours_Per_Night	\
0	1	1	6.5	
1	7	0	7.5	
2	6	1	5.0	
3	11	0	7.0	
4	0	1	6.0	

	Mental_Health_Score	Relationship_Status	Conflicts_Over_Social_Media	\
0	6	1	3	
1	8	2	0	
2	5	0	4	
3	7	2	1	
4	6	1	2	

	Addicted_Score
0	8
1	3
2	9
3	4
4	7

FEATURE ENGINEERING

```
X["Usage_Sleep_Ratio"] = (  
    X["Avg_Daily_Usage_Hours"] /  
    X["Sleep_Hours_Per_Night"])  
# Another feature  
X["Mental_Conflict_Score"] = (  
    X["Mental_Health_Score"] +  
    X["Conflicts_Over_Social_Media"])  
print("Feature Engineering Completed!")
```

New meaningful features were created to improve model performance, and the dataset was divided into training and testing sets.

TRAIN TEST SPLIT

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)  
print("Training data shape:", X_train.shape)  
print("Testing data shape:", X_test.shape)
```

Training data shape: (561, 14)
Testing data shape: (141, 14)

FEATURE SCALING

```
# Scaling makes all values similar in size
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

LINEAR REGRESSION MODEL

```
lr_model = LinearRegression()
# Train model
lr_model.fit(X_train_scaled, y_train)
print("Linear Regression model trained!")
Linear Regression model trained!
```

PREDICTIONS

```
lr_pred = lr_model.predict(X_test_scaled)
print(lr_pred[:10])
```

```
[6.59781579 7.65206075 7.09724118 6.82658814 6.83038933 6.5958222
 6.81017839 7.99020943 4.91801581 5.98956709]
```

Feature scaling standardized the data, and Linear Regression was trained to predict addiction scores.

EVALUATION OF LINEAR REGRESSION

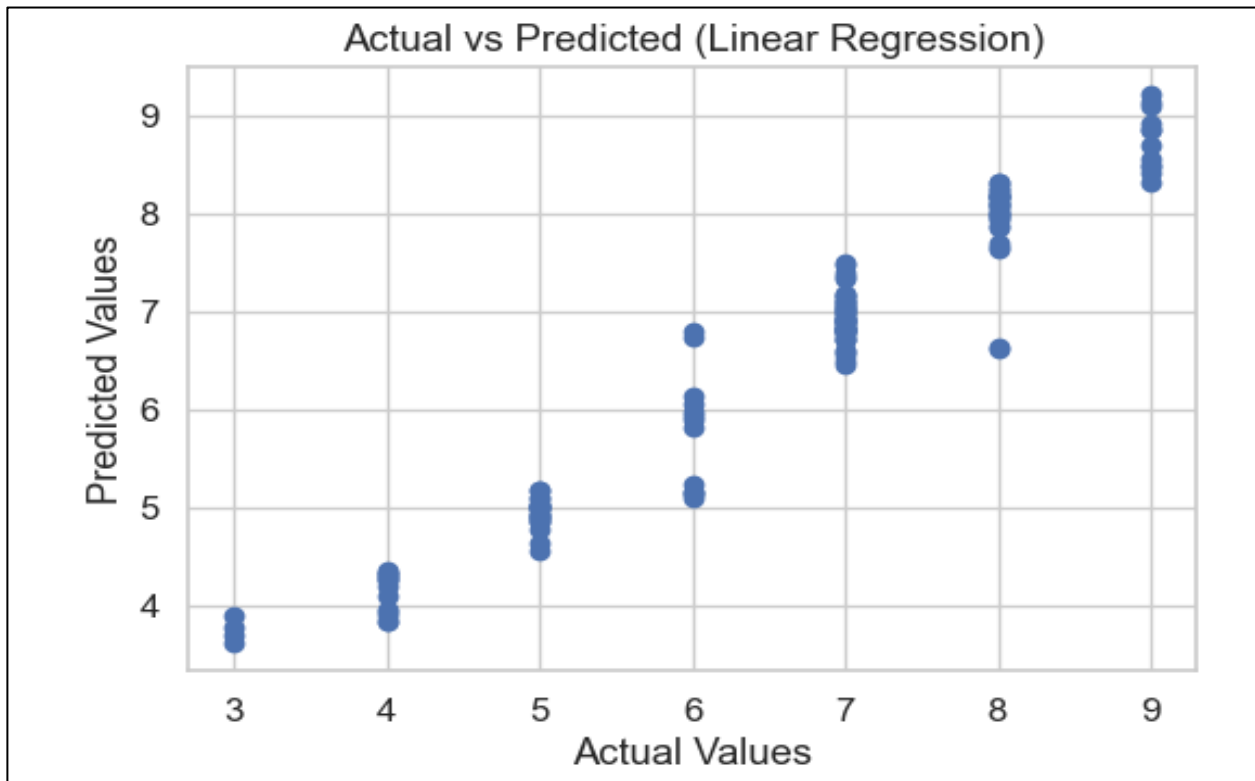
```
lr_mae = mean_absolute_error(y_test, lr_pred)
lr_mse = mean_squared_error(y_test, lr_pred)
lr_rmse = np.sqrt(lr_mse)
lr_r2 = r2_score(y_test, lr_pred)
print("LINEAR REGRESSION RESULTS")
print("MAE :", lr_mae)
print("MSE :", lr_mse)
print("RMSE :", lr_rmse)
print("R2 Score :", lr_r2)
```

```
LINEAR REGRESSION RESULTS
MAE : 0.24568803538967685
MSE : 0.12480334012589848
RMSE : 0.3532751620562907
R2 Score : 0.9483553574839109
```

The Linear Regression model achieved strong performance with high R^2 score and low prediction error.

ACTUAL VS PREDICTED GRAPH

```
plt.figure(figsize=(6,4))
plt.scatter(y_test, lr_pred)
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Actual vs Predicted (Linear Regression)")
plt.show()
```



The predicted values are close to actual values, indicating good model accuracy.

RANDOM FOREST MODEL

```
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
# Train model
rf_model.fit(X_train, y_train)
print("Random Forest model trained!")
```

Random Forest model trained!

RANDOM FOREST EVALUATION

```
rf_mae = mean_absolute_error(y_test, rf_pred)
rf_mse = mean_squared_error(y_test, rf_pred)
rf_rmse = np.sqrt(rf_mse)
rf_r2 = r2_score(y_test, rf_pred)
print("RANDOM FOREST RESULTS")
print()
print("MAE :", rf_mae)
print("MSE :", rf_mse)
print("RMSE :", rf_rmse)
print("R2 Score :", rf_r2)
```

RANDOM FOREST RESULTS

MAE : 0.06801418439716307
MSE : 0.036698581560283675
RMSE : 0.19156873847338368
R2 Score : 0.984813826908667

Random Forest achieved better prediction accuracy compared to Linear Regression.

FEATURE IMPORTANCE

Mental Health Score and Conflicts Over Social Media are the most important features affecting addiction score prediction.

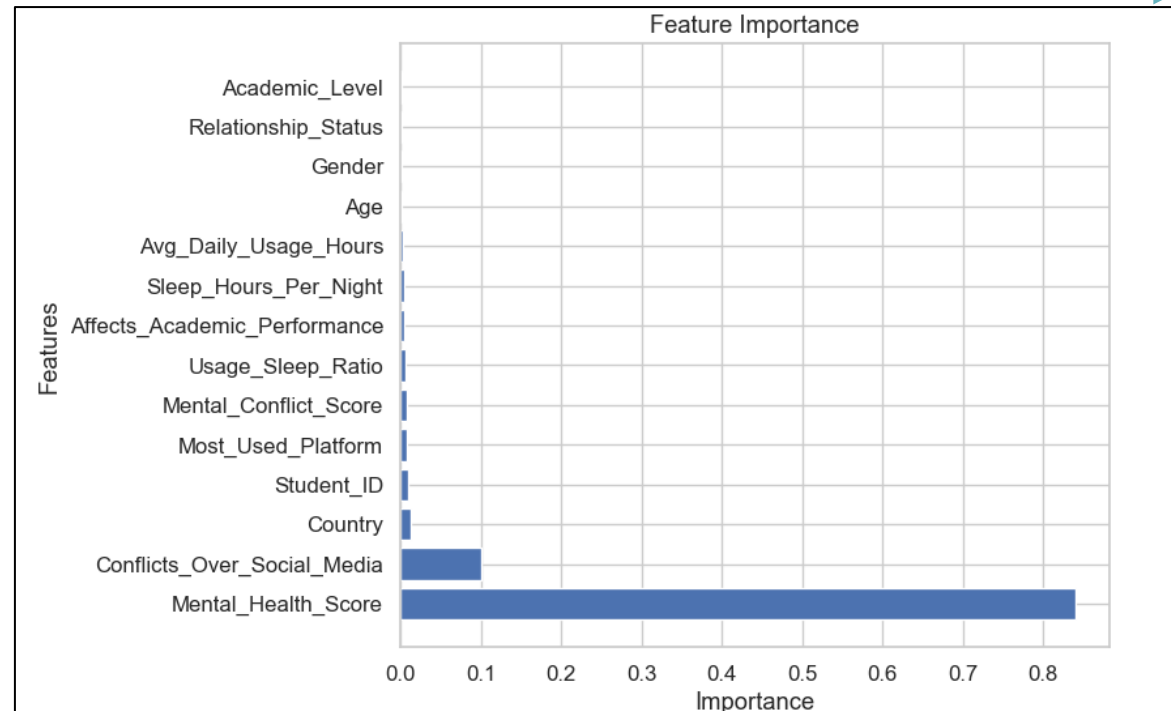
Which features are most responsible for predicting addiction score?

```
importance = rf_model.feature_importances_  
feature_names = X.columns  
importance_df = pd.DataFrame({  
    "Feature": feature_names,  
    "Importance": importance})  
importance_df = importance_df.sort_values(  
    by="Importance",  
    ascending=False)  
print(importance_df)
```

	Feature	Importance
9	Mental_Health_Score	0.839996
11	Conflicts_Over_Social_Media	0.100868
4	Country	0.013045
0	Student_ID	0.009908
6	Most_Used_Platform	0.007684
13	Mental_Conflict_Score	0.007191
12	Usage_Sleep_Ratio	0.006626
7	Affects_Academic_Performance	0.004797
8	Sleep_Hours_Per_Night	0.004478
5	Avg_Daily_Usage_Hours	0.002383
1	Age	0.001304
2	Gender	0.000601
10	Relationship_Status	0.000574
3	Academic_Level	0.000546

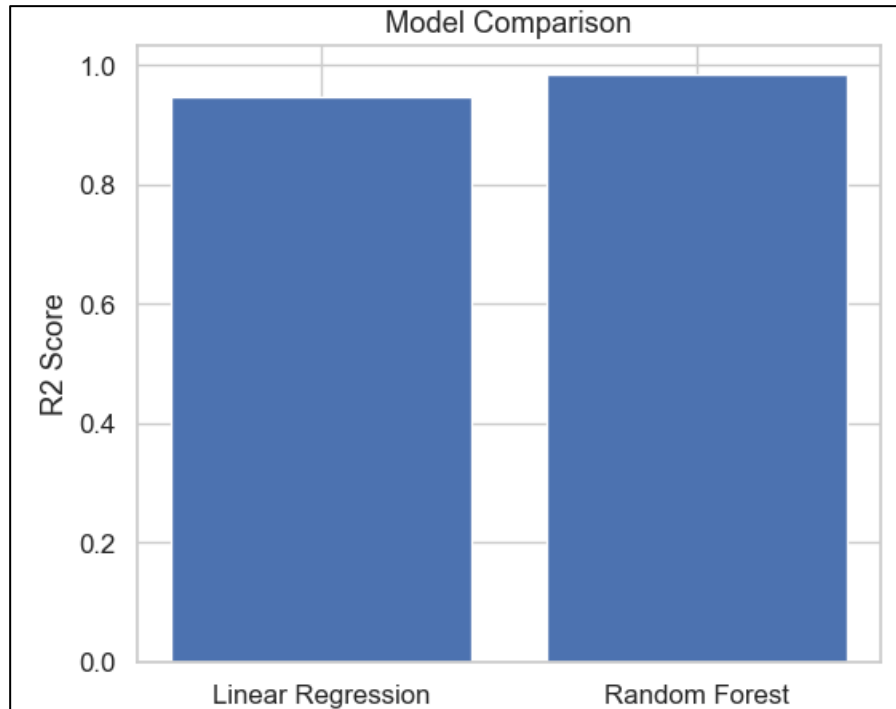
FEATURE IMPORTANCE GRAPH

```
plt.figure(figsize=(10,6))  
plt.barh(importance_df["Feature"],importance_df["Importance"])  
plt.xlabel("Importance")  
plt.ylabel("Features")  
plt.title("Feature Importance")  
plt.show()
```



MODEL COMPARISON

```
models = ["Linear Regression", "Random Forest"]  
r2_scores = [lr_r2, rf_r2]  
plt.figure(figsize=(8,5))  
plt.bar(models, r2_scores)  
plt.ylabel("R2 Score")  
plt.title("Model Comparison")  
plt.show()
```



FINAL RESULT

```
print("FINAL RESULTS")  
print("Linear Regression R2 Score :", lr_r2)  
print("Random Forest R2 Score :", rf_r2)  
if rf_r2 > lr_r2:  
    print("Random Forest performed better!")  
else:  
    print("Linear Regression performed better!")
```

FINAL RESULTS

```
Linear Regression R2 Score : 0.9483553574839109  
Random Forest R2 Score : 0.984813826908667  
Random Forest performed better!
```

Random Forest outperformed Linear Regression and was selected as the best-performing model for this project.

Conclusion

- Successfully analyzed students' social media addiction patterns using Machine Learning.
- Mental Health Score and Conflicts Over Social Media were identified as the most important factors affecting addiction levels.
- Performed data cleaning, EDA, feature engineering, encoding, scaling, and model evaluation.
- Compared Linear Regression and Random Forest models for prediction.
- Random Forest performed better with higher accuracy and R^2 Score.
- The project demonstrates how Machine Learning can help analyze real-world social issues and predict behavioral patterns.

Thank you

